Relatório Técnico — Integração e Validação da Extração de Features em Hardware

1. Objetivo

Este documento descreve o processo de integração e validação funcional dos módulos responsáveis pela extração de features de tráfego TCP/IP em hardware, contemplando as atividades:

- **2.4 — Integração Hash Unit + Controller**
- **2.5 — Integração Controller + Flow Finalizer**

O objetivo foi verificar se os módulos conseguem trabalhar em conjunto, desde a identificação do fluxo por meio do cálculo de hash até a disponibilização do vetor de features.

A validação foi realizada por simulação no Vivado/XSim, utilizando um testbench de integração com 181 pacotes de entrada e verificações dos resultados produzidos pelo sistema.

Além da verificação funcional, foram analisadas as limitações da arquitetura atual, principalmente em relação à diferença de velocidade entre a Hash Unit e o Controller e à ausência de um mecanismo de controle de fluxo entre esses módulos.

---

2. Arquitetura da integração

A arquitetura integrada pode ser dividida em três etapas principais:

1. **Hash Unit:** recebe os dados do pacote e calcula o CRC-16 utilizado como índice de identificação do fluxo.
2. **Controller:** utiliza o índice e a chave do fluxo para consultar a Flow Table, classificar o pacote e atualizar os registros armazenados.
3. **Flow Finalizer:** recebe os registros válidos durante a varredura da Flow Table e os converte em vetores de features de 248 bits.

O fluxo geral de processamento é:

flowchart TD

A["Dados do pacote TCP/IP"] --> B["Hash Unit"]

B -->|"hash_index + flow_key + metadados + hash_valid"| C["Controller"]

C <-->|"rd_en, rd_addr, rd_data, rd_valid"| D["Flow Table"]

C -->|"flow_entry[383:0] + fin_valid"| E["Flow Finalizer"]

E -->|"fin_done"| C

E -->|"feature_vector[247:0]"| F["Saída de features"]

G["pcap_done"] --> C

A Flow Table armazena os registros dos fluxos identificados. O Controller realiza as operações de consulta, classificação, atualização e varredura dessa memória.

Ao receber o sinal pcap_done, o Controller inicia a varredura da tabela para disponibilizar os registros válidos ao Flow Finalizer.

2.1. Organização dos registros na Flow Table

Cada entrada lógica da Flow Table possui 16 palavras de 32 bits, totalizando 512 bits.

A organização utilizada é:

Palavra	**Conteúdo**	**Largura**
W0	Indicador de validade (valid)	32 bits
W1	Parte da chave do fluxo	32 bits
W2	Parte da chave do fluxo	32 bits
W3	Parte da chave do fluxo	32 bits
W4	Parte da chave do fluxo	32 bits
W5	packet_count	32 bits
W6	byte_count	32 bits
W7	Primeira parte do timestamp inicial	32 bits
W8	Segunda parte do timestamp inicial	32 bits
W9	Terceira parte do timestamp inicial	32 bits
W10	Primeira parte do timestamp final	32 bits
W11	Segunda parte do timestamp final	32 bits
W12	Terceira parte do timestamp final	32 bits
W13–W15	Reservadas	96 bits

A chave do fluxo possui 104 bits. Os timestamps inicial e final possuem 80 bits cada.

A entrada lógica ocupa 512 bits, mas a interface da memória utiliza palavras físicas de 32 bits. Por isso, cada entrada é acessada por meio de 16 endereços consecutivos.

Considerando 65.536 entradas lógicas:

- Quantidade de entradas: 65.536.
- Palavras por entrada: 16.
- Total de palavras físicas: 1.048.576.
- Largura de cada palavra: 32 bits.

- Capacidade lógica total: 4 MiB.

2.2. Organização do vetor de registro entregue ao Finalizer

O Controller entrega ao Flow Finalizer um registro de 384 bits, composto pelas palavras W1 a W12.

A composição é:

Campo	Intervalo de bits	Largura
flow_key	[103:0]	104 bits
packet_count	[159:128]	32 bits
byte_count	[191:160]	32 bits
first_timestamp	[271:192]	80 bits
last_timestamp	[383:288]	80 bits

Os campos intermediários são organizados em palavras de 32 bits, conforme a disposição da Flow Table.

O indicador de validade W0 não faz parte do registro de 384 bits. Ele é utilizado pelo Controller para identificar quais entradas devem ser processadas durante a varredura.

---

3. Atividade 2.4 — Integração Hash Unit + Controller

3.1. Objetivo

A atividade 2.4 teve como objetivo integrar a saída da Hash Unit à entrada do Controller, garantindo que os dados calculados fossem recebidos corretamente e utilizados no processamento dos fluxos.

Foram considerados os seguintes aspectos:

- Conexão dos sinais entre os módulos.
- Correspondência das larguras dos sinais.
- Captura do índice de hash e dos metadados.
- Funcionamento da FSM do Controller.
- Classificação dos pacotes como MISS, HIT ou COLLISION.
- Comportamento da interface sem sinal ready.

3.2. Sinais utilizados na integração

A Hash Unit fornece ao Controller os seguintes sinais:

Sinal	Largura	Função

hash_index	16 bits	Índice calculado pelo CRC-16, utilizado para acessar a entrada correspondente na Flow Table.
flow_key	104 bits	Identificação do fluxo a partir dos campos utilizados na composição da chave.
packet_length	16 bits	Comprimento do pacote recebido.
timestamp	80 bits	Timestamp associado ao pacote.
hash_valid	1 bit	Indica que o resultado da Hash Unit está disponível para captura pelo Controller.

O sinal hash_valid é responsável por indicar a disponibilidade dos dados. Quando o Controller está no estado IDLE e identifica hash_valid ativo, inicia o processamento do pacote.

Na implementação atual, não existe um sinal ready para indicar que o Controller está disponível para receber novos resultados.

3.3. Funcionamento da FSM do Controller

O Controller utiliza uma máquina de estados finitos (FSM) para controlar o processamento dos pacotes.

De maneira simplificada, a sequência de processamento é:

1. **IDLE:** aguarda um resultado válido da Hash Unit.
2. **LOOKUP_REQ:** solicita a leitura do indicador de validade da entrada correspondente.
3. **LOOKUP_WAIT:** aguarda o retorno da leitura.
4. **CLASSIFY:** determina se o pacote corresponde a um fluxo inexistente, existente ou a uma colisão.
5. **HIT_READ_REQ / HIT_READ_WAIT:** quando ocorre HIT, realiza as leituras necessárias para recuperar os contadores existentes.
6. **PREPARE_WRITE:** prepara os dados que serão armazenados.
7. **WRITE_REQ / WRITE_WAIT:** solicita e aguarda a conclusão das operações de escrita.
8. **WAIT_VALID_LOW:** aguarda a desativação de hash_valid antes de aceitar outro resultado.

Durante a classificação, o Controller utiliza três resultados possíveis:

Resultado	**Código**	**Comportamento**
MISS	2'b01	A posição está livre. Um novo registro de fluxo é criado.
HIT	2'b10	A posição contém o mesmo fluxo. Os contadores e o timestamp final são atualizados.

COLLISION	2'b11	A posição contém outro fluxo. O registro não é sobrescrito e o contador de colisões é incrementado.

O resultado lookup_result é utilizado no testbench para acompanhar a classificação de cada pacote.

O sinal lookup_done indica que o processamento do pacote foi concluído pelo Controller. Ele é utilizado como referência para contabilizar e verificar os resultados da classificação.

O Controller também possui o sinal collision_count, que registra a quantidade de colisões. Sua implementação utiliza saturação para evitar que o contador ultrapasse seu valor máximo.

3.4. Diferença de velocidade entre Hash Unit e Controller

Um dos principais pontos identificados durante a análise da integração foi a diferença de velocidade entre os módulos.

A Hash Unit calcula o CRC-16 em aproximadamente 13 ciclos de processamento, publicando o resultado por meio de hash_valid.

Já o Controller precisa consultar a memória e, dependendo do resultado, executar diferentes operações:

- Para MISS, verificar a posição e escrever o novo registro.
- Para HIT, realizar leituras dos contadores existentes e atualizar o registro.
- Para COLLISION, identificar a incompatibilidade e contabilizar a ocorrência.

Portanto, o tempo necessário para processar um pacote no Controller depende do caminho percorrido pela FSM e da latência das operações na Flow Table.

Limitação da interface atual

A interface entre os módulos não possui sinal ready nem FIFO.

Isso significa que o Controller não informa diretamente à Hash Unit quando está disponível para receber outro resultado.

Se a Hash Unit publicar um novo resultado enquanto o Controller estiver ocupado, esse resultado poderá ser ignorado, pois o Controller somente captura novos dados quando está no estado IDLE.

Essa característica representa uma limitação importante da arquitetura atual.

Estratégia adotada no teste

Para validar inicialmente o caminho completo sem modificar os módulos que já haviam sido testados individualmente, foi adotada uma estratégia de processamento sequencial.

O testbench envia um pacote, aguarda a conclusão do processamento pelo Controller e, em seguida, respeita um intervalo de 80 ciclos antes de prosseguir com o próximo pacote.

Essa estratégia permite validar a integração funcional sem provocar sobreposição de resultados na interface.

É importante destacar que esse intervalo representa uma condição controlada de teste. Ele não comprova que a arquitetura consegue receber continuamente pacotes na taxa máxima de processamento da Hash Unit.

3.5. Casos de teste da atividade 2.4

Foram utilizados 181 pacotes de entrada com dados previamente conhecidos.

O testbench acompanhou o processamento e verificou a classificação realizada pelo Controller.

Os resultados observados foram:

Classificação	**Quantidade**
MISS	179
HIT	1
COLLISION	1
Total	**181**

Interpretação dos resultados

MISS — 179 ocorrências

Foram identificados 179 casos em que o índice consultado correspondia a uma posição livre da Flow Table. Nesses casos, o Controller realizou a criação dos registros de fluxo.

HIT — 1 ocorrência

Foi identificado um caso em que o índice e a chave correspondiam a um fluxo já armazenado. O Controller realizou o caminho de atualização do registro existente.

COLLISION — 1 ocorrência

Foi identificado um caso em que o índice de hash correspondia a uma posição ocupada por outro fluxo. O Controller identificou a colisão e evitou sobrescrever o registro existente.

A ocorrência dos três resultados demonstra que o testbench conseguiu exercitar os diferentes caminhos de classificação implementados na FSM.

3.6. Conclusão da atividade 2.4

A integração Hash Unit + Controller foi validada funcionalmente.

Os testes demonstraram que os resultados da Hash Unit são encaminhados e capturados pelo Controller, que consegue utilizar os dados recebidos para consultar a Flow Table e classificar os pacotes nos cenários de MISS, HIT e COLLISION.

A estratégia de envio sequencial permitiu validar o comportamento sem modificar o protocolo original entre os módulos.

Entretanto, a ausência de controle de fluxo entre Hash Unit e Controller permanece como uma limitação para aplicações que exigem maior vazão de processamento.

---

4. Atividade 2.5 — Integração Controller + Flow Finalizer

4.1. Objetivo

A atividade 2.5 teve como objetivo integrar o caminho de disponibilização dos registros de fluxo ao Flow Finalizer, verificando a interpretação dos campos armazenados e a geração correta do vetor de features.

Foram avaliados:

- A conexão entre Controller e Finalizer.
- O protocolo de comunicação sem sinal ready.
- A correspondência entre os campos do registro de 384 bits e sua interpretação.
- A extração dos contadores e timestamps.
- O cálculo da duração do fluxo.
- A composição do vetor de features de 248 bits.
- O processamento de múltiplos registros consecutivos.

4.2. Sinais utilizados na integração

A interface entre Controller e Flow Finalizer utiliza os seguintes sinais:

Sinal	**Largura**	**Função**
flow_entry	384 bits	Registro de fluxo disponibilizado pelo Controller.
fin_valid	1 bit	Indica que o registro está disponível para processamento.
fin_done	1 bit	Indica que o Finalizer concluiu o processamento do registro.
feature_vector	248 bits	Vetor de features gerado pelo Finalizer.
pcap_done	1 bit	Indica que o recebimento dos pacotes foi concluído e solicita a varredura da Flow Table.

O protocolo implementado não utiliza sinal ready.

O Controller disponibiliza o registro mantendo fin_valid ativo até receber a confirmação de processamento por meio de fin_done.

O Finalizer identifica a primeira ocorrência de fin_valid, processa o registro e gera um pulso em fin_done.

O mecanismo interno fin_seen impede que o mesmo registro seja processado repetidamente enquanto fin_valid permanecer ativo.

Após a conclusão, o Controller retira fin_valid e aguarda seu retorno ao nível baixo antes de disponibilizar outro registro.

4.3. Funcionamento da varredura da Flow Table

Depois que todos os pacotes são enviados, o testbench ativa pcap_done.

O Controller identifica a solicitação e inicia a varredura da Flow Table.

A sequência simplificada é:

1. O Controller verifica o indicador de validade W0 de uma entrada.
2. Se W0 indicar que a entrada é válida, o Controller lê as palavras W1 a W12.
3. Os dados são organizados no vetor flow_entry[383:0].
4. O Controller ativa fin_valid.
5. O Flow Finalizer processa o registro e gera fin_done.
6. O Controller aguarda o término do sinal de confirmação.
7. Se configurado, o Controller invalida a entrada por meio da escrita em W0.
8. O Controller avança para a próxima entrada.

Quando encontra uma entrada inválida, o Controller não precisa encaminhá-la ao Finalizer e continua a varredura.

A implementação utiliza a configuração INVALIDATE_AFTER_SEND para controlar a invalidação após a entrega do registro.

4.4. Interpretação dos 384 bits pelo Finalizer

O Flow Finalizer recebe o vetor de 384 bits e extrai os campos correspondentes.

A organização utilizada pelo módulo é:

Campo	**Seleção no flow_entry**	**Largura**
flow_key	[103:0]	104 bits
packet_count	[159:128]	32 bits
byte_count	[191:160]	32 bits
first_timestamp	{[271:256], [255:192]}	80 bits
last_timestamp	{[367:352], [351:288]}	80 bits

Os timestamps são reconstruídos a partir das palavras de 32 bits armazenadas na Flow Table.

Essa organização é importante porque qualquer diferença entre a posição dos campos no Controller e a interpretação do Finalizer poderia produzir features incorretas, mesmo que o protocolo de comunicação estivesse funcionando.

4.5. Cálculo da duração

O Finalizer calcula a duração do fluxo por meio da diferença entre o timestamp final e o timestamp inicial.

[
duration = last_timestamp - first_timestamp
]

O cálculo é realizado utilizando 80 bits, conforme a largura dos timestamps.

A duração representa o intervalo entre o primeiro e o último pacote registrado para aquele fluxo.

É importante observar que o cálculo atual utiliza aritmética unsigned de 80 bits. Portanto, os timestamps precisam estar representados em uma escala e unidade consistentes, e o timestamp final deve ser posterior ou igual ao inicial dentro do intervalo considerado.

4.6. Composição do vetor de features

O vetor de saída possui 248 bits e é organizado da seguinte maneira:

Campo	**Intervalo de bits**	**Largura**
flow_key	[247:144]	104 bits
packet_count	[143:112]	32 bits
byte_count	[111:80]	32 bits
duration	[79:0]	80 bits
Total	**[247:0]**	**248 bits**

A composição é:

[
feature_vector =
{flow_key,\ packet_count,\ byte_count,\ duration}
]

Assim, o vetor reúne a identificação do fluxo, a quantidade de pacotes, a quantidade de bytes e a duração observada.

Essa saída constitui a representação utilizada para as etapas posteriores de análise ou classificação por modelos de Machine Learning.

4.7. Casos de teste da atividade 2.5

O testbench acompanhou a varredura da Flow Table e comparou as features produzidas com os valores esperados para os registros válidos.

Foram verificadas as seguintes condições:

- Correspondência entre o registro de 384 bits e os campos interpretados pelo Finalizer.
- Correção da chave do fluxo.
- Correção de packet_count.
- Correção de byte_count.

- Correção da duração calculada.
- Largura de 248 bits do vetor de saída.
- Processamento dos registros válidos.
- Funcionamento do handshake entre Controller e Finalizer.

O resultado da primeira varredura foi:

179 features conferidas com sucesso.

O número de features corresponde às 179 entradas únicas e válidas geradas pelos casos MISS. O caso HIT atualizou uma dessas entradas, enquanto o caso COLLISION não criou um novo registro.

4.8. Verificação de invalidação

Após o processamento dos registros, o testbench verificou se as entradas enviadas haviam sido invalidadas na Flow Table.

A invalidação é realizada por meio da escrita no campo W0, responsável por indicar se a entrada está válida.

O resultado observado foi:

Todas as entradas enviadas foram invalidadas com sucesso.

Essa verificação é importante para evitar que registros já processados sejam enviados novamente em uma nova varredura.

4.9. Segunda varredura

Depois da invalidação, o testbench solicitou uma segunda varredura da Flow Table.

O objetivo foi verificar se os registros processados anteriormente seriam encaminhados novamente ao Finalizer.

O resultado foi:

Segunda varredura concluída sem repetição de features.

Essa etapa validou o comportamento conjunto da varredura, da invalidação e do controle de entrega de registros.

4.10. Conclusão da atividade 2.5

A integração Controller + Flow Finalizer foi validada funcionalmente.

A simulação demonstrou que o Controller consegue disponibilizar registros de 384 bits, o Finalizer interpreta os campos esperados e o vetor de features de 248 bits é produzido corretamente para os registros avaliados.

Também foi verificado o funcionamento do protocolo fin_valid / fin_done, a entrega de múltiplos registros, a invalidação das entradas e a ausência de repetição na segunda varredura.

---

5. Explicação do testbench de integração

5.1. Objetivo do testbench

O testbench tb_top_extracao_features_com_entrada foi desenvolvido para verificar o comportamento integrado do sistema, utilizando a entrada de pacotes e observando os resultados gerados pelos módulos internos.

Diferentemente dos testes unitários, que verificam um módulo isoladamente, o teste de integração avalia a comunicação e o comportamento conjunto dos componentes.

O testbench utiliza a instância do módulo principal:

top_extracao_features_com_entrada

Dessa forma, os módulos Hash Unit, Controller, Flow Table e Flow Finalizer são exercitados por meio da interface superior do sistema.

5.2. Carregamento dos vetores de teste

O testbench utiliza o arquivo:

hash_vectors.mem

Esse arquivo contém os vetores previamente preparados para a simulação.

Cada vetor possui 216 bits, distribuídos da seguinte forma:

Intervalo	**Conteúdo**	**Largura**
[215:112]	flow_key	104 bits
[111:96]	packet_length	16 bits
[95:16]	timestamp	80 bits
[15:0]	CRC-16 esperado	16 bits

A leitura é realizada por meio de $readmemh, que carrega os valores do arquivo hexadecimal para a memória de vetores do testbench.

Isso permite utilizar entradas conhecidas e comparar o resultado produzido pela Hash Unit com o valor esperado.

5.3. Geração do clock e reset

O testbench gera o clock utilizado pelos módulos síncronos.

O reset é aplicado inicialmente para colocar o sistema em um estado conhecido.

Após a liberação do reset, os módulos começam a operar normalmente.

Essa etapa garante que os testes sejam iniciados com a FSM do Controller e os sinais de controle em condições definidas.

5.4. Envio dos pacotes

O testbench percorre os 181 vetores carregados.

Para cada pacote, são aplicados os campos correspondentes na entrada do módulo principal:

- Endereço IP de origem.
- Endereço IP de destino.
- Porta de origem.
- Porta de destino.
- Protocolo.
- Comprimento do pacote.
- Timestamp.
- Sinal de validade da entrada.

A Hash Unit processa os dados e disponibiliza o resultado para o Controller.

O testbench acompanha hash_valid e verifica o resultado do cálculo de hash e os metadados correspondentes.

Após a publicação do resultado, o testbench aguarda a conclusão do processamento pelo Controller.

O intervalo de 80 ciclos entre pacotes é utilizado para evitar a sobreposição de resultados, considerando que a interface atual não possui sinal ready nem FIFO.

5.5. Verificação da classificação

O testbench acompanha o sinal interno lookup_done do Controller.

Quando esse sinal indica a conclusão do processamento de um pacote, o testbench consulta lookup_result e contabiliza a classificação correspondente:

- MISS.
- HIT.
- COLLISION.

Essa verificação permite confirmar se o Controller percorreu os caminhos esperados da FSM.

Também é possível acompanhar o contador interno collision_count para verificar se as colisões foram contabilizadas corretamente.

O uso de sinais internos por referência hierárquica facilita a observação do comportamento do Controller durante a simulação.

Entretanto, esse método cria dependência entre o testbench e a organização interna do módulo. Uma futura evolução pode utilizar sinais de status disponibilizados por uma interface de verificação específica.

5.6. Solicitação da varredura

Depois que todos os pacotes são processados, o testbench ativa pcap_done.

Esse sinal informa ao Controller que a entrada de pacotes foi finalizada e que a Flow Table pode ser varrida.

O testbench acompanha o sinal scan_done para identificar o término da varredura.

Durante esse processo, o monitor observa os pulsos de fin_done e verifica os vetores de features produzidos.

5.7. Comparação das features

Para cada registro esperado, o testbench monta o vetor de referência com:

- Chave do fluxo.
- Quantidade de pacotes.
- Quantidade de bytes.
- Duração esperada.

O vetor produzido pelo DUT é comparado com a referência.

Essa comparação é fundamental porque não basta verificar que fin_done foi ativado. Também é necessário garantir que os dados efetivamente entregues estejam corretos.

A verificação considera a ordem dos registros e a posição esperada de cada feature na saída.

5.8. Verificação da invalidação

Após a primeira varredura, o testbench verifica as posições correspondentes às entradas processadas na memória da Flow Table.

O objetivo é confirmar que o indicador de validade foi removido.

Essa verificação utiliza acesso hierárquico à memória interna da Flow Table, conforme a implementação utilizada no projeto.

Como depende do nome e da organização interna da memória, essa parte do testbench deverá ser atualizada caso o módulo de memória seja modificado.

5.9. Segunda varredura

Por fim, o testbench solicita uma nova varredura.

Como as entradas anteriores foram invalidadas, não devem existir novos registros válidos correspondentes aos pacotes já processados.

O testbench verifica que nenhuma feature é produzida novamente.

Essa etapa valida a prevenção de duplicidade na entrega dos registros.

---

6. Resultado geral da simulação

A execução do testbench foi concluída com sucesso.

Os resultados observados foram:

Métrica	**Resultado**

Pacotes enviados	181
MISS	179
HIT	1
COLLISION	1
Features conferidas	179
Entradas invalidadas	Todas as entradas enviadas
Segunda varredura	Sem features repetidas

6.1. Testes aprovados

Teste	**Resultado**
Hash Unit + Controller	PASSOU
Controller + Flow Table	PASSOU
Controller + Finalizer	PASSOU
Layout dos 384 bits	PASSOU
Vetor de features de 248 bits	PASSOU
Handshake fin_valid / fin_done	PASSOU
Invalidação da Flow Table	PASSOU
Segunda varredura sem repetição	PASSOU

A simulação foi encerrada com $finish no tempo informado pelo Vivado:

4231870 ns

O resultado demonstra que os cenários contemplados pelo testbench foram concluídos sem falhas.

É importante ressaltar que a aprovação é referente à validação funcional em simulação. Ela não representa, por si só, uma comprovação de síntese, temporização ou funcionamento físico na FPGA.

---

7. Limitações identificadas e oportunidades de melhoria

Embora a integração tenha sido aprovada, alguns pontos merecem atenção para as próximas etapas do desenvolvimento.

7.1. Ausência de FIFO ou controle de fluxo entre Hash Unit e Controller

Este é o principal ponto de atenção da arquitetura atual.

A Hash Unit possui uma latência de aproximadamente 13 ciclos para o cálculo do CRC-16, enquanto o Controller pode precisar de diversas operações de leitura e escrita na Flow Table.

Como a interface não possui ready, o Controller não consegue informar à Hash Unit que está ocupado.

Se um resultado for publicado enquanto o Controller estiver processando outro pacote, ele poderá ser perdido.

Alternativa 1 — Manter o processamento sequencial

A primeira alternativa é manter a estratégia utilizada no teste:

1. Enviar um pacote.
2. Aguardar a conclusão do processamento pelo Controller.
3. Aguardar a liberação da interface.
4. Enviar o próximo pacote.

Essa abordagem é simples e adequada para validar o funcionamento funcional da arquitetura.

Sua principal desvantagem é limitar a vazão do sistema, pois a Hash Unit não pode operar continuamente sem considerar a disponibilidade do Controller.

Alternativa 2 — Implementar uma FIFO

Uma alternativa mais robusta é adicionar uma FIFO entre a Hash Unit e o Controller.

A FIFO armazenaria temporariamente os resultados calculados pela Hash Unit enquanto o Controller estivesse ocupado.

A interface poderia ser organizada com sinais como:

Sinal	**Função**
fifo_wr_en	Solicita a escrita de um resultado na FIFO.
fifo_full	Indica que a FIFO está cheia.
fifo_empty	Indica que não existem resultados disponíveis.
fifo_rd_en	Solicita a leitura de um resultado.
fifo_data	Transporta os dados armazenados.

O resultado armazenado precisaria conter todos os campos necessários ao Controller:

hash_index + flow_key + packet_length + timestamp

A FIFO permitiria desacoplar temporalmente a Hash Unit e o Controller.

Entretanto, seria necessário definir o comportamento da Hash Unit quando a FIFO estivesse cheia. Uma solução seria introduzir um sinal de controle para impedir a publicação de novos resultados até que exista espaço disponível.

Alternativa 3 — Implementar handshake ready/valid

Outra possibilidade é modificar o protocolo para utilizar dois sinais:

- valid: indica que o produtor possui dados válidos.
- ready: indica que o consumidor pode receber os dados.

A transferência ocorre quando ambos estão ativos simultaneamente no mesmo ciclo de clock.

Esse protocolo permite que o Controller controle diretamente a aceitação dos resultados.

Entretanto, para funcionar corretamente, a Hash Unit precisa manter os dados estáveis enquanto valid estiver ativo e aguardar a aceitação pelo consumidor.

Recomendação: manter a interface atual durante a validação funcional inicial e avaliar a implementação de uma FIFO ou handshake ready/valid em uma etapa posterior, caso a aplicação exija maior vazão.

7.2. Ausência de controle de fluxo na entrada da Hash Unit

A limitação de vazão não está restrita à interface entre Hash Unit e Controller.

Também é importante verificar como os pacotes chegam à Hash Unit.

Caso a entrada permita que novos pacotes sejam enviados sem considerar a disponibilidade interna do módulo, poderá existir uma limitação semelhante na primeira etapa do processamento.

Uma melhoria seria definir claramente o protocolo de entrada, incluindo:

- Sinal de validade dos dados recebidos.
- Condição de aceitação de um novo pacote.
- Comportamento quando a Hash Unit estiver ocupada.
- Estabilidade dos metadados durante o processamento.

Essa análise deve considerar a interface completa, desde a entrada do pacote até a atualização da Flow Table.

7.3. Dependência de uma janela de espera fixa no testbench

O testbench utiliza um intervalo de 80 ciclos entre os pacotes.

Essa estratégia é adequada para o teste funcional atual, mas não representa necessariamente o tempo mínimo necessário para cada transação.

Uma melhoria seria utilizar um mecanismo de sincronização baseado em eventos.

Por exemplo:

- Aguardar lookup_done.

- Verificar se hash_valid retornou ao nível baixo.
- Aguardar a condição de disponibilidade do Controller.
- Somente então iniciar a próxima transação.

Dessa forma, o testbench ficaria menos dependente de um número fixo de ciclos.

Também seria possível medir o tempo de processamento de cada pacote e identificar a diferença de latência entre MISS, HIT e COLLISION.

7.4. Ampliação dos testes de colisão

O teste atual demonstrou uma ocorrência de COLLISION.

Como o comportamento de colisão é importante para a integridade da Flow Table, seria interessante ampliar os cenários.

Sugestões:

- Testar várias colisões consecutivas.
- Verificar se o registro original permanece inalterado.
- Verificar se collision_count é incrementado corretamente.
- Testar o limite de saturação do contador.
- Verificar o comportamento após uma colisão seguida de um MISS ou HIT.

Isso permitiria avaliar melhor a robustez da implementação.

7.5. Ampliação dos testes de HIT

O teste atual apresentou um único caso de HIT.

Seria interessante adicionar sequências maiores de pacotes pertencentes ao mesmo fluxo.

Por exemplo:

1. Primeiro pacote do fluxo: MISS.
2. Segundo pacote do mesmo fluxo: HIT.
3. Terceiro pacote do mesmo fluxo: HIT.
4. Quarto pacote do mesmo fluxo: HIT.

O testbench poderia verificar se:

- packet_count é incrementado corretamente.
- byte_count acumula os comprimentos dos pacotes.
- first_timestamp permanece inalterado.
- last_timestamp recebe o timestamp do pacote mais recente.

Esse cenário ajudaria a verificar a atualização progressiva dos registros.

7.6. Verificação de limites e overflow dos contadores

Os campos packet_count e byte_count possuem 32 bits.

Em uma aplicação de maior duração, é importante avaliar o comportamento quando esses contadores atingirem seus limites.

Deve-se definir se a implementação utilizará:

- Aritmética modular, permitindo overflow.
- Saturação no valor máximo.
- Algum mecanismo de atualização ou reinicialização.

A mesma análise deve ser realizada para os timestamps e para o contador de colisões.

Essas condições não foram comprovadas pelo resultado atual da simulação e devem ser tratadas como cenários adicionais.

7.7. Verificação da duração dos fluxos

O Finalizer calcula a duração pela diferença entre os timestamps final e inicial.

É importante avaliar cenários adicionais:

- Fluxo com apenas um pacote.
- Fluxo com duração igual a zero.
- Fluxo com timestamps muito próximos.
- Fluxo com timestamps próximos ao limite de representação.
- Possível inconsistência ou inversão dos timestamps.

Também deve ser documentada a unidade utilizada nos timestamps, como ciclos, microssegundos ou outra referência temporal.

A unidade da duração será fundamental para a interpretação posterior das features pelo modelo de Machine Learning.

7.8. Escalabilidade da Flow Table

A Flow Table possui 65.536 entradas lógicas.

Cada entrada ocupa 512 bits, resultando em aproximadamente 4 MiB de armazenamento.

É importante avaliar se essa organização é adequada ao dispositivo FPGA escolhido.

Além da capacidade total, devem ser considerados:

- Quantidade de blocos de memória disponíveis.
- Largura e profundidade suportadas pelas memórias físicas.
- Latência de leitura e escrita.
- Possibilidade de inferência de BRAM.
- Impacto da implementação na frequência máxima.

- Recursos utilizados para o controle da varredura.

A simulação funcional não comprova que toda a memória será implementada em blocos de memória internos da FPGA. Essa confirmação depende da síntese e dos relatórios de implementação.

7.9. Tempo de varredura da Flow Table

O Controller percorre as entradas da Flow Table para identificar os registros válidos.

Como a implementação considera 65.536 entradas, a varredura completa pode exigir uma quantidade significativa de ciclos.

O tempo depende da latência das operações de leitura, da quantidade de entradas válidas e do protocolo de entrega ao Finalizer.

Uma melhoria seria medir:

- Ciclos totais da varredura.
- Ciclos gastos por entrada inválida.
- Ciclos gastos por entrada válida.
- Ciclos de espera do handshake.
- Tempo total até scan_done.

Essas informações ajudam a avaliar a escalabilidade do sistema.

7.10. Dependência de sinais internos no testbench

O testbench utiliza sinais internos do DUT, como:

- lookup_done.
- lookup_result.
- collision_count.
- scan_done.
- Memória interna da Flow Table.

Essa estratégia facilita a verificação detalhada do comportamento da arquitetura.

Entretanto, torna o testbench dependente da organização interna dos módulos. Uma alteração de nome ou estrutura pode exigir mudanças no testbench, mesmo que a interface externa continue igual.

Como evolução, seria interessante disponibilizar uma interface específica de verificação, com sinais de status e eventos de conclusão.

Também é possível considerar SystemVerilog Assertions (SVA) para verificar propriedades do protocolo e da FSM.

---

8. Melhorias prioritárias

As melhorias podem ser organizadas de acordo com sua importância para a evolução do projeto.

Prioridade	Melhoria	Objetivo
Alta	Implementar FIFO ou handshake entre Hash Unit e Controller	Evitar perda de resultados e permitir maior vazão.
Alta	Ampliar os testes de HIT	Verificar corretamente a atualização dos contadores e timestamps.
Alta	Verificar preservação do registro original em colisões	Garantir integridade dos dados armazenados.
Média	Substituir intervalos fixos de espera por sincronização por eventos	Tornar o testbench mais robusto.
Média	Medir o tempo de processamento e de varredura	Avaliar desempenho e latência.
Média	Testar overflow e limites dos contadores	Verificar comportamento em condições extremas.
Média	Ampliar os testes de timestamps	Garantir consistência da duração calculada.
Média	Verificar inferência de memória na síntese	Confirmar viabilidade da Flow Table na FPGA.
Baixa	Desenvolver uma interface de verificação independente	Reduzir dependência de sinais internos do DUT.
Baixa	Adicionar propriedades SVA	Automatizar a verificação de protocolos e estados.

Anexo A — Resumo dos principais sinais

Módulo / Interface	Sinal	Largura	Descrição
Entrada do sistema	in_valid	1	Indica pacote válido na entrada.
Entrada do sistema	pcap_done	1	Solicita o início da varredura após o envio dos pacotes.
Hash Unit → Controller	hash_index	16	Índice CRC-16 do fluxo.
Hash Unit → Controller	flow_key	104	Chave de identificação do fluxo.

Hash Unit → Controller	packet_length	16	Comprimento do pacote.
Hash Unit → Controller	timestamp	80	Timestamp do pacote.
Hash Unit → Controller	hash_valid	1	Indica resultado de hash disponível.
Controller → Flow Table	rd_en	1	Habilita leitura.
Controller → Flow Table	rd_addr	20	Endereço da palavra a ser lida.
Flow Table → Controller	rd_data	32	Palavra lida da memória.
Flow Table → Controller	rd_valid	1	Indica dado de leitura válido.
Controller → Flow Table	wr_en	1	Habilita escrita.
Controller → Flow Table	wr_addr	20	Endereço da palavra a ser escrita.
Controller → Flow Table	wr_data	32	Dado a ser armazenado.
Flow Table → Controller	wr_done	1	Indica conclusão da escrita.
Controller → Finalizer	flow_entry	384	Registro completo disponibilizado para extração.
Controller → Finalizer	fin_valid	1	Indica registro disponível.
Finalizer → Controller	fin_done	1	Indica conclusão do processamento.
Finalizer → Saída	feature_vector	248	Vetor de features produzido.

Anexo B — Observação sobre o escopo da validação

Os resultados deste documento se referem à simulação funcional realizada com o testbench e os vetores de entrada utilizados.

Não devem ser interpretados como comprovação de:

- Funcionamento físico na FPGA.

- Frequência máxima de operação.
- Ausência de violações de temporização.
- Consumo final de recursos de hardware.
- Capacidade de processar continuamente todos os pacotes sem perda.
- Cobertura de todas as combinações possíveis de pacotes e estados internos.

Esses aspectos dependem de testes adicionais, síntese, implementação e validação em hardware.